FreeRTOS 移植笔记 — STM32F103C8 / Keil MDK

> 日期：2026-09-19
> 项目：EnvMon_Voice (t2)

一、基本信息

项	值
FreeRTOS 版本	FreeRTOS-Kernel **V11.3.1**
源码包	`FreeRTOSv202604.01-LTS`（LTS 长期支持版）
源码路径	`E:\EnvMon_Voice\FreeRTOSv202604.01-LTS\FreeRTOS-LTS\FreeRTOS\FreeRTOS-Kernel`
目标路径	`E:\EnvMon_Voice\t2\freertos`
MCU	STM32F103C8（Cortex-M3，20KB RAM / 64KB Flash）
编译器	ARM Compiler **V5.06 update 7 (build 960)**，即 AC5
系统时钟	HSE 8MHz × PLL9 = **72MHz**

二、文件清单

共转移 **32 个文件**，全部与官方源码逐字节一致（用 `diff` 校验）。

`freertos/src/` — 内核源文件（7 个）

```
croutine.c  event_groups.c  list.c  queue.c
stream_buffer.c  tasks.c  timers.c
```

`freertos/inc/` — 头文件（21 个 + 配置文件）

```
FreeRTOS.h  task.h  queue.h  semphr.h  timers.h  event_groups.h
stream_buffer.h  message_buffer.h  list.h  croutine.h  portable.h
projdefs.h  deprecated_definitions.h  atomic.h  stack_macros.h
StackMacros.h  mpu_wrappers.h  mpu_prototypes.h  mpu_syscall_numbers.h
newlib-freertos.h  picolibc-freertos.h
FreeRTOSConfig.h   ← 配置文件，从模板改写而来
```

`freertos/port/` — 端口层（3 个）

文件	来源	说明
`port.c`	`portable/RVDS/ARM_CM3/`	Cortex-M3 端口实现
`portmacro.h`	`portable/RVDS/ARM_CM3/`	端口宏定义
`heap_4.c`	`portable/MemMang/`	内存管理方案 4

三、关键决策及理由

3.1 为什么选 `RVDS/ARM_CM3` 而不是 `GCC/ARM_CM3` 或 `Keil/ARM_CM3`

源码包里 `portable/` 下有多个候选目录，容易选错：

- **`Keil/`** — **不是端口**，里面只有一个 `See-also-the-RVDS-directory.txt`，内容是 "Nothing to see here."，实际指向 RVDS
- **`ARMClang/`** — **也不是端口**，只有一个 `Use-the-GCC-ports.txt`，指向 GCC
- **`RVDS/ARM_CM3`** — 使用 `__asm { }` 内嵌汇编语法，是 **AC5（armcc）** 用的
- **`GCC/ARM_CM3`** — 使用 GCC 风格 `__asm volatile`，是 **AC6（armclang）** 用的

工程文件 `project.uvprojx` 中 `<uAC6>0</uAC6>`，即使用 **AC5**，因此正确选择是 `RVDS/ARM_CM3`。

> 判断依据：如果改用 AC6 编译器，需要换成 `GCC/ARM_CM3` 的 `port.c` / `portmacro.h`。

3.2 为什么选 `heap_4.c`

官方提供 5 种内存管理方案：

方案	特点	适用性
heap_1	只能分配不能释放	太简单，不适用
heap_2	可释放，但不合并碎片	有碎片风险
heap_3	包装标准 `malloc` / `free`	需要 C 库堆，占用大
heap_4	**可释放 + 相邻空闲块合并**	**最通用，推荐**
heap_5	同 heap_4，支持多块不连续内存	本项目内存连续，不需要

`heap_4.c` 支持碎片合并，是 STM32 项目的通用选择。

3.3 时基（Tick）安排 —— 一个容易忽略的点

FreeRTOS 需要一个周期性中断作为系统节拍。Cortex-M 上有两种做法：

- 用 SysTick（FreeRTOS 默认）
- 用普通定时器

本项目已有的 `Core/Src/stm32f1xx_hal_timebase_tim.c` 已经把 HAL 的时基放在 TIM4 上（见该文件第 74 行 `htim4.Instance = TIM4`）。

这正好是理想状态：

用途	定时器
HAL 库时基（`HAL_IncTick`、`HAL_Delay`）	TIM4
FreeRTOS 系统节拍	SysTick

两者互不干扰。如果 HAL 时基仍用 SysTick，就会和 FreeRTOS 抢占同一个中断，导致 `HAL_Delay` 与 RTOS 节拍冲突。**这一点本项目已经天然满足，无需改动。**

四、`FreeRTOSConfig.h` 修改明细

配置文件取自官方模板 `FreeRTOS-Kernel/examples/template_configuration/FreeRTOSConfig.h`（675 行），
针对 STM32F103 修改了以下各项。

配置项	模板原值	修改为	理由
`configCPU_CLOCK_HZ`	20000000	**72000000**	模板默认 20MHz 是 QEMU 仿真值；实际 HSE 8MHz × PLL9 = 72MHz

配置项	模板原值	修改为	理由
`configTICK_RATE_HZ`	100	**1000**	100Hz 节拍太粗（10ms），STM32 常规用 1ms
`configUSE_TIME_SLICING`	0	**1**	开启同优先级任务的轮转调度
`configTOTAL_HEAP_SIZE`	4096	**10240**	4KB 不够任务+队列使用；20KB RAM 下取 10KB 折中
`configSUPPORT_STATIC_ALLOCATION`	1	**0**	只用 heap_4 动态分配；设为 1 会要求实现 `vApplicationGetIdleTaskMemory` 等回调
`configKERNEL_INTERRUPT_PRIORITY`	0	`15 << (8-4)` = 0xF0	**必须非 0**。设为 0 表示最高优先级，会让内核中断抢占一切
`configMAX_SYSCALL_INTERRUPT_PRIORITY`	0	`5 << (8-4)` = 0x50	**必须非 0**，否则 `port.c` 直接 `#error` 编译失败
`configMAX_API_CALL_INTERRUPT_PRIORITY`	0	同上	`configMAX_SYSCALL_INTERRUPT_PRIORITY` 的别名，两个都要改
`configCHECK_FOR_STACK_OVERFLOW`	2	**0**	设为 2 会要求实现 `vApplicationStackOverflowHook`，否则链接报错。调试期可改回 1 或 2 并补上该函数

新增的辅助宏

```
#define configPRIO_BITS                          4
#define configLIBRARY_LOWEST_INTERRUPT_PRIORITY  15
#define configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY  5
```

STM32F103 的 Cortex-M3 实现了 **4 个优先级位**（16 级）。优先级值需要左移 `(8 - 4) = 4` 位，
因为 Cortex-M 的 NVIC 只使用寄存器高 4 位。

五、踩坑记录

坑 1：中断处理函数重名（编译期报重复符号）

现象：`Core/Src/stm32f1xx_it.c` 中已存在 3 个 CubeMX 生成的中断处理函数
（下面是当时的简写，行号也是**当时**的）：

```
void SVC_Handler(void)    { }   /* 第 145 行 */
void PendSV_Handler(void) { }   /* 第 171 行 */
void SysTick_Handler(void){ }   /* 第 184 行 */
```

而 FreeRTOS 端口层也要实现这 3 个中断，导致链接期重复符号。

预期之外的细节：FreeRTOS **V11.3.1 的 RVDS 端口不再自带名称映射宏**了。
在 `freertos/` 整个目录下搜索 `SVC_Handler` —— **零命中**。
`port.c` 定义的实际函数名是 `vPortSVCHandler`、`xPortPendSVHandler`、`xPortSysTickHandler`（见 port.c 第 215、405、444 行）。

所以**只注释掉 `it.c` 里的函数是不够的** —— 那样向量表会指向未定义的符号。
两头都必须改：

① 在 `FreeRTOSConfig.h` 中补上映射宏（`freertos/inc/FreeRTOSConfig.h:348-350`）：

```
#define vPortSVCHandler     SVC_Handler
#define xPortPendSVHandler  PendSV_Handler
#define xPortSysTickHandler SysTick_Handler
```

② 在 `stm32f1xx_it.c` 中用 `#if 0` 停用那三个空函数
（`Core/Src/stm32f1xx_it.c:153`、`:183`、`:202`；用 `#if 0` 而非删除，便于日后对照）

坑 2：AC5 内嵌汇编无法解析 `UL` 后缀（编译错误 A1586E）

现象：首次编译报错

```
..\freertos\port\port.c(424): error: A1586E: Bad operand types
(UnDefOT, Constant) for operator
```

原因：`port.c` 第 424 行是一句**内嵌汇编**：

```
mov r0, #configMAX_SYSCALL_INTERRUPT_PRIORITY
```

我最初把它定义为 `( 5UL << ( 8UL - 4UL ) )`。
在 **C 代码**里 `5UL` 完全合法，但在 **AC5 的内嵌汇编器**里，
`UL` 后缀无法识别 —— 汇编器把 `5UL` 当成未定义符号（UnDefOT = Undefined Operand Type），
于是报"操作数类型错误"。

修复：改用 STM32CubeMX 的标准写法 —— 纯十进制字面量，不带任何后缀：

```
#define configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY    5    /* 不是 5UL */
#define configMAX_SYSCALL_INTERRUPT_PRIORITY \
        ( configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY << ( 8 - configPRIO_BITS ) )
```

展开后汇编器看到的是 `#( 5 << ( 8 - 4 ) )`，可以正常求值。

> **经验**：凡是会被展开进 `__asm { }` 的宏，都不能带 `UL` / `U` / `L` 后缀。

坑 3：工程文件路径与文件实际位置不一致

`project.uvprojx` 中引用的是 `..\freertos\inc\FreeRTOSConfig.h`，
但配置文件最初被放在了 `freertos\` 根目录。**Keil 中会显示为缺失文件**。
已将 `FreeRTOSConfig.h` 移入 `freertos/inc/` 对齐。

> 注：`inc/` 和工程根目录都在 include 路径中（`..\freertos;..\freertos\inc`），
> 所以编译器两种位置都能找到，但工程文件树必须与实际路径一致。

六、验证记录

已完成的验证

1. **文件完整性** — 32 个文件全部用 `diff` 与官方源码比对，**逐字节一致**
2. **工程引用** — 脚本校验 `project.uvprojx` 中 32 个引用路径**全部存在**
   - `freertos/inc`：22 个
   - `freertos/port`：3 个
   - `freertos/src`：7 个
3. **编译结果** — Keil 命令行构建通过：

```
Program Size: Code=8040 RO-data=312 RW-data=92 ZI-data=2020
"project\project.axf" - 0 Error(s), 0 Warning(s).
```

4. **堆链接情况** — map 文件确认 `heap_4.o` 因无引用被整段移除，属正常现象，详见第七章第 2 条

七、待办事项

✅ 1. `vTaskStartScheduler()` 尚未调用 → 现在已调用（当时是待办）

> **当时的进度记录**：写到这一节时工程**能编译链接，但 FreeRTOS 并不会真正运行**，
> 需要在 `Core/Src/main.c` 中，于外设初始化完成之后、 `while(1)` 之前调用：
>
> ```
> /* 创建任务 ... */
> vTaskStartScheduler();   /* 启动调度器，正常情况下不会返回 */
>
> /* 调度器启动失败才会走到这里 */
> while (1) { }
> ```
>
> —— 那之后已经补上了，调度器**现在正常运行**。

实际写法（`Core/Src/main.c:102-108`，逐字）：

```
App_Init();

vTaskStartScheduler();

/* 只有 heap 不足、连空闲任务都建不起来时，vTaskStartScheduler() 才会返回 */
printf("ERROR: vTaskStartScheduler() returned!\r\n");
Error_Handler();
```

任务创建全部收进了 `App_Init()`（`APP/app_main.c`）， `main.c` 里只留这一句调用 —— 跟当时想的"在 main.c 里直接 `xTaskCreate` "不一样，但起点是同一个。

✅ 2. `ZI-data` 未包含 10KB 堆 —— 已查明，属正常现象

`configTOTAL_HEAP_SIZE` 设为 10240 字节， `ucHeap` 应位于 `.bss` 计入 `ZI-data`，
但实测 `Total RW Size` 仅 2112 字节，堆不在其中。

已通过 map 文件确认原因：链接器的未引用段移除机制把整个 `heap_4.c` 回收了。
map 的镜像组成表里 `heap_4.o` 的贡献是 `Code 0 / RO 0 / RW 0 / ZI 0`，
并在移除清单中明确列出：

```
Removing heap_4.o(i.pvPortMalloc), (324 bytes).
Removing heap_4.o(i.prvHeapInit), (68 bytes).
Removing heap_4.o(i.vPortFree), (148 bytes).
...
```

因为当时 `main.c` 里还没调用任何 FreeRTOS API， `pvPortMalloc` 无人引用，
于是连 `ucHeap[10240]` 一起被丢弃。**这不是问题** —— 一旦调用
`vTaskStartScheduler()`，内核和堆就会被真正链接进来，届时 RAM 占用约为
2112 + 10240 ≈ 12.4KB / 20KB。

> **补充（任务跑起来之后）**：上面的推断已经应验。 `heap_4.o` 现在被整个链了进来
> （map 里只剩 `pvPortCalloc` 、 `vPortGetHeapStats` 这几个没人用的函数被移除），
> 当前构建是 `Code=18720 RO-data=524 RW-data=164 ZI-data=12828`，
> `Total RW Size` **12.69KB** —— 和 12.4KB 的估算对得上。

> **顺带验证了一件重要的事**：map 中 `port.o` 的 Code 为 194 字节，且有
> `startup_stm32f103xb.o(RESET) refers to port.o(.emb_text) for SVC_Handler`
> —— 说明向量表确实指向了 FreeRTOS 端口的实现，**handler 映射宏配置正确**。

3. 其他可选项

- `configCHECK_FOR_STACK_OVERFLOW` 当前为 0（关闭）。调试期建议改为 2，并在 `main.c` 中补上：

```
void vApplicationStackOverflowHook(TaskHandle_t xTask, char *pcTaskName) {
    taskDISABLE_INTERRUPTS();
    for (;;);
}
```

- 优先级分组需确保为 `NVIC_PRIORITYGROUP_4` （STM32 HAL 默认即是），
  否则 `configMAX_SYSCALL_INTERRUPT_PRIORITY = 5 << 4` 的含义会变化
- `configTOTAL_HEAP_SIZE = 10240` 是按 20KB RAM 取的折中值，可按实际任务数量调整
  （见 `freertos/inc/FreeRTOSConfig.h` 第 290 行附近）

八、⚠️ CubeMX 重新生成的坑

每次在 CubeMX 里点 "Generate Code"，下面这些改动都会被冲掉：

文件	会被冲掉的内容
`Core/Src/stm32f1xx_it.c`	三个空 handler 的 `#if 0` 保护 → 编译重新报重复符号
`Core/Src/dma.c`	`DMA1_Channel6_IRQn` 优先级 5 → 被改回 0

应对办法：

- `it.c` —— 目前只能手工重新加回 `#if 0` （见第五章坑 1）
- `dma.c` —— 已经在 `project.ioc` 里同步改了
  （`NVIC.DMA1_Channel6_IRQn=true\:5\:0\:...`），CubeMX 重新生成时会保留 5

> 补充：`.ioc` 中已确认 `NVIC.PriorityGroup=NVIC_PRIORITYGROUP_4` 和
> `NVIC.TimeBase=TIM4_IRQn`，这两项印证了第四章的优先级取值和第三章 3.3 的时基安排。

九、ESP8266 驱动优化

对 `BSP/ESP8266/esp8266.c` 做了安全性优化，**未改变驱动架构和对外接口**。

> ⚠️ 这句只管**当时那一轮安全性优化**。后来接收路径换成了 DMA 环形缓冲
> （见下面"未做的优化"），架构和对外接口都动过了 —— 现在看代码时以代码为准。

> 路径说明：当时这个文件在 `WIFI/ESP8266/` 下，后来整个驱动搬到了 `BSP/ESP8266/`
> （`WIFI/` 目录现在已经是个空目录）。本文其余地方仍按当时的写法提到 `WIFI/`。

修复的问题

#	位置	问题	修复
1	`SetWiFiMode` / `ConnectWiFi` / `TCPConnect` / `TCPSend`	`sprintf` 写入固定长度栈数组， 无边界检查，SSID/密码过长会溢出	改用 `snprintf(cmd, sizeof(cmd), ...)`
2	文件头部全局变量	`ESP8266_RxBuf` / `ESP8266_RxLen` 非 static， 污染命名空间	改为 `static`，外部经已有的 `ESP8266_GetRxBuf()` / `GetRxBufLen()` 访问
3	`ESP8266_TCPConnect`	`CIPSTART` 前未确保单连接模式	增加 `AT+CIPMUX=0`
4	接收等待逻辑	缓冲区满时返回 `ESP8266_ERR_TIMEOUT`， 明明收到数据却报超时	新增状态码 `ESP8266_ERR_BUF_FULL`；并调整判断 （**先判响应再判满**， 否则最后一个字节恰好凑齐响应时会把成功误报
5	`ESP8266_TCPSend`	用 `SendCmd("")` 等响应，而 `HAL_UART_Transmit` 在 `Size==0` 时返回 `HAL_ERROR`，该调用实为空操作	抽出 `ESP8266_WaitResponse()`，只等响应不发数

#	位置	问题	修复
6	`ConnectWiFi` / `TCPConnect` / `TCPSend` / `GetIPAddress`	无入参校验	增加 NULL / 边界检查，返回 `ESP8266_ERR_PARAM`（该状态码原先定义了但从
7	`Core/Src/dma.c`	`DMA1_Channel6_IRQn` 优先级为 0，高于 `configMAX_SYSCALL_INTERRUPT_PRIORITY` (5)，FreeRTOS 下该 ISR 不能调用任何 RTOS API	优先级改为 5，并在 `.ioc` 中同步

✅ 当时"未做的优化"（性能相关）—— 已于 2026-09-20 做掉

> **当时的记录**：工程**已经为 USART2 配好了 DMA 接收**
> （usart.c:146-159， `DMA1_Channel6` ， `__HAL_LINKDMA` ），
> 但驱动完全没用它 —— 接收仍是 `HAL_UART_Receive` 逐字节阻塞轮询，
> 每次空读要等 10ms，512 字节的响应要轮询 512 次。
> HAL 1.1.10 支持 `HAL_UARTEx_ReceiveToIdle_DMA` ，当时想的是
> "DMA 环形缓冲 + **空闲中断**"方案，能大幅降低 CPU 占用，但会改变驱动架构
> 且必须上硬件实测 AT 交互，所以那次没做。

实际落地的方案和预想的不一样，这里要说清楚：

	当时预想	最终实现
DMA 模式	环形	**环形**（ `DMA_CIRCULAR` ， `Core/Src/usart.c:152` ） ✅ 一致
怎么知道"收了多少"	IDLE 空闲中断	**轮询 `CNDTR` 剩余计数**（ `esp8266.c:79-107` ）
USART2 中断	必须使能（优先级 5）	**一个都不开**
与 RTOS 的交互	中断里 `xSemaphoreGiveFromISR` + `portYIELD_FROM_ISR`	**无信号量、无 `FromISR`**

为什么最后不用 IDLE 中断 —— `BSP/ESP8266/esp8266.c:26-30` 的原文理由：

> 为什么不用 IDLE 中断判"一帧收完了"?
> 那要多开一个 USART2 中断, 还得纠结"中断里能不能调 FromISR 的 RTOS API"
> —— 一碰 RTOS, BSP 就破了"不认识 FreeRTOS"这条规矩(见 bsp_delay.h).
> 轮询的代价只是最多 1ms 的发现延迟, 而 AT 响应本身都是几十毫秒级的,
> 完全够用. 少一个中断, 少一堆麻烦.

代价是"最多 1ms 的发现延迟"，对几十毫秒级的 AT 应答完全够用；换来的是 **BSP 层不依赖 RTOS** —— 和 `BSP_DelayMs()` 的弱/强符号方案（见《FreeRTOS驱动适配笔记》3.1）是同一个设计取向。

配合这一点， `ESP8266_WaitResponse()` 里的空档期也从"逐字节忙等"改成了 `BSP_DelayMs(1)` （任务里即 `vTaskDelay` ），CPU 不再被占满。

十、修改过的文件一览

文件	修改内容
`freertos/` （32 个文件）	从官方源码复制（新增）
`freertos/inc/FreeRTOSConfig.h`	从模板改写：9 项配置 + 3 个优先级辅助宏 + 3 个 handler 映射宏（新增）
`Core/Src/stm32f1xx_it.c`	三个空中断函数用 `#if 0` 停用
`Core/Src/dma.c`	`DMA1_Channel6_IRQn` 优先级 0 → 5
`project.ioc`	`NVIC.DMA1_Channel6_IRQn` 优先级 0 → 5（与 dma.c 同步）
`BSP/ESP8266/esp8266.c`	安全性优化，见第九章（**当时代码在 `WIFI/ESP8266/` ，后来搬到 `BSP/ESP8266/`** ）

文件	修改内容
BSP/ESP8266/esp8266.h	新增 ESP8266_ERR_BUF_FULL 、 ESP8266_WaitResponse() 、 ESP8266_TX_TIMEOUT （当时还列了 ESP8266_RX_POLL_MS ，**该宏已随后来的 DMA 改造删除**；后来又陆续加了 ESP8266_DMA_BUF_SIZE 、 ESP8266_ScanAP() 、 ESP8266_TCPRecvRaw() 、 ESP8266_FACTORY_RESET_ON_BOOT 等）

编译验证

```
Program Size: Code=8040 RO-data=312 RW-data=92 ZI-data=2020
"project\project.axf" - 0 Error(s), 0 Warning(s).
```

> 这是**当时那一步**（只有内核、还没有任何任务）的镜像大小。
> 任务跑起来之后内存占用会涨到 12KB 量级 —— 见第七章第 2 条的补充。